

Du notebook au projet que tu peux montrer



7 étapes, chacune avec son livrable, son erreur fréquente et son check avant de partager.

Un notebook qui tourne n'est pas un projet. C'est ce que je vois le plus souvent en relisant des portfolios de candidats en reconversion : du code propre, zéro erreur d'exécution, et pourtant rien à montrer. Le recruteur n'ouvre presque jamais le code en premier. Il ouvre le lien, regarde ce qui s'affiche, et décide en quelques secondes s'il continue à lire.

Du code au dépôt structuré.

Ici, l'objectif est simple : que n'importe qui ouvre ton dossier et comprenne où regarder, sans que tu sois là pour l'expliquer.

1. Structurer le dépôt

Livrable : Une arborescence claire : `data/`, `src/`, `notebooks/`, `README.md`. Chaque dossier a un rôle unique et prévisible, lisible en 10 secondes sans contexte.

```
mon-projet/  
  data/      -> échantillon ou données publiques  
  src/       -> scripts et fonctions réutilisables  
  notebooks/ -> exploration, analyse, graphiques  
  README.md  -> contexte, question, résultat
```

Erreur fréquente

Tout à la racine : `notebooks`, fichiers CSV et scripts mélangés dans le même dossier, sans hiérarchie.

Check avant de partager

Ouvre le dépôt sans rien expliquer à côté. Si la structure ne se comprend pas seule, elle n'est pas prête.

2. Extraire un script réutilisable

Livrable : Une fonction ou un petit module `.py`, séparé de l'exploration du notebook, que tu peux appeler et tester seul.

```
# src/clean_data.py  
import pandas as pd  
  
def nettoyer_colonnes(df):  
    df = df.dropna(subset=["date", "montant"])  
    df["date"] = pd.to_datetime(df["date"])  
    return df  
  
if __name__ == "__main__":  
    print(nettoyer_colonnes(pd.read_csv("data/exemple.csv")).head())
```

Erreur fréquente

Copier coller la même logique de nettoyage dans trois notebooks différents, avec de petites variations qui finissent par se contredire.

Check avant de partager

Exécute le script seul, hors notebook (`python src/clean_data.py`). S'il plante sur un import manquant ou un chemin de fichier codé en dur, il n'est pas prêt.

3. Nettoyer le notebook

Livable : Un notebook qui s'exécute de haut en bas sans erreur après un `Restart & Run All` : ordre d'exécution linéaire, pas de dépendance cachée à une cellule lancée hors ordre.

Erreur fréquente

Cellules dans le désordre, code mort laissé en commentaire, sortie d'erreur encore affichée sous une cellule.

Check avant de partager

Kernel > Restart & Run All. Zéro erreur, zéro cellule vide oubliée, zéro sortie qui ne correspond plus au code.

À retenir : le code propre ne suffit pas encore

Un dépôt bien rangé et un notebook qui tourne prouvent que tu sais coder, pas que tu sais raconter un projet. C'est le chapitre suivant qui fait la différence.

Du dépôt au récit lisible.

Ici, la technique est déjà là. Ce qui manque, c'est la raison pour laquelle quelqu'un devrait s'y intéresser.

4. Documenter : un README qui raconte le projet

Livable : Un README avec trois choses : le contexte (pourquoi ce projet existe), la question posée, et le résultat trouvé. Pas seulement une notice d'installation.

Erreur fréquente

Un README qui commence par `pip install -r requirements.txt` sans jamais dire pourquoi le projet existe ni ce qu'il a trouvé.

Check avant de partager

Fais lire le README à quelqu'un hors data pendant 30 secondes. Il doit pouvoir dire à quelle question le projet répond.

5. Fournir des données d'exemple

Livable : Un jeu de données publiques ou anonymisées, assez réaliste pour que l'analyse garde du sens, prêt à être rejoué par quelqu'un d'autre.

Erreur fréquente, éliminatoire

Pousser un extrait réel non anonymisé : un fichier RH avec noms et salaires, un export client avec adresses email. Ça disqualifie le portfolio immédiatement, pas seulement la note de style.

Check avant de partager

Relis chaque colonne du fichier une par une. Aucune donnée identifiante ou confidentielle ne doit rester, même dans une colonne annexe oubliée.

De la vitrine à l'ouverture recruteur.

Ici, on quitte le contenu du projet pour regarder ce qui se passe au moment précis où quelqu'un clique sur ton lien.

6. Choisir un hébergement stable et accessible

Livable : Un lien public qui s'ouvre sans compte ni installation. GitHub couvre le dépôt de code : public, gratuit, et là depuis assez longtemps pour qu'un lien tienne.

Pour le rendu du notebook et la mini-app, choisis toi-même avec trois critères : le service affiche ton notebook sans demander de compte au lecteur, le lien reste valide sans que tu redéploies, et le plan gratuit ne réclame pas de carte bancaire. Vérifie-les le jour où tu publies, pas le jour où tu as mémorisé le nom du service.

Erreur fréquente

Garder le projet uniquement en local, ou l'envoyer en zip par email : le recruteur doit alors télécharger, dézipper, installer, avant même de voir le résultat.

Check avant de partager

Ouvre le lien en navigation privée, sans être connecté. Il doit s'afficher immédiatement, sans demande de compte.

7. Vérifier ce que le recruteur ouvre en premier

Livable : La confirmation de ce qui s'affiche en premier quand on clique le lien, avant tout contexte : c'est ça, la vraie page de garde du projet.

Erreur fréquente, éliminatoire

Le lien pointe directement dans un dossier de notebooks bruts, jamais rendu, sans README visible : le recruteur voit du JSON de cellules avant de voir une seule phrase de contexte.

Check avant de partager

Ouvre le lien à froid, depuis un autre appareil ou en navigation privée, sans rien dire avant. Regarde ce qui s'affiche en premier, et demande-toi si ça donne envie de continuer.

À retenir : le projet est jugé sur ce qui est ouvert, pas sur ce qui est écrit

Le lien, la première image, la première phrase du README : c'est ça la vitrine. Pas le nombre de lignes de code, pas la complexité du modèle. Si la première seconde ne donne rien à lire, le reste ne sera jamais ouvert.

Fait main, en solo.

Si ce guide t'a aidé à faire le tri dans ton portfolio, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

